

BUILD-APIS.md

Version : V1

Statut : Référence officielle de construction des APIs

Type : Build Blueprint

1. Objectif

Ce document définit les règles officielles de conception, d'implémentation, de sécurisation et d'évolution des APIs de Vitala.

Les contrats fonctionnels sont définis dans **16-VITALA-API-V1.md**.

Le présent document définit **la manière dont ces contrats doivent être implémentés**.

2. Principes fondamentaux

Toutes les APIs doivent respecter les principes suivants :

- API First
- Domain First
- Stateless
- Security by Design
- Versioning Ready
- Documentation First
- Test First

Aucune API ne peut être développée sans contrat officiel.

3. Architecture des APIs

Chaque domaine possède ses propres APIs.

Exemple :

- Auth API
- UDI API
- Flash API
- Mission API
- Trust API
- Media API
- Sync API
- Intelligence APIs

- Platform APIs

Chaque API appartient exclusivement à son domaine.

4. Responsabilités

Une API est responsable de :

- recevoir une requête ;
- valider les données ;
- authentifier l'utilisateur ;
- vérifier les permissions ;
- appeler les cas d'utilisation ;
- retourner une réponse conforme au contrat.

Une API ne contient jamais la logique métier.

5. Cycle de traitement

Chaque requête suit obligatoirement les étapes suivantes :

1. Authentification.
2. Vérification des permissions.
3. Validation des données.
4. Exécution du cas d'utilisation.
5. Journalisation si nécessaire.
6. Construction de la réponse.

Aucune étape ne peut être ignorée.

6. Validation

Toute API doit valider :

- les types de données ;
- les champs obligatoires ;
- les formats ;
- les limites métier ;
- les règles de sécurité.

Les données invalides sont rejetées avant toute logique métier.

7. Réponses

Les réponses doivent être :

- cohérentes ;
- documentées ;
- déterministes ;
- compatibles avec les contrats officiels.

Une réponse ne doit jamais exposer des informations internes.

8. Gestion des erreurs

Toutes les erreurs doivent être :

- typées ;
- documentées ;
- traçables ;
- cohérentes entre les domaines.

Les erreurs techniques internes ne doivent jamais être retournées aux clients.

9. Authentification

Toutes les APIs protégées utilisent Supabase Auth.

Les identités métier sont représentées par l'UDI.

L'authentification technique et l'identité métier restent séparées.

10. Autorisations

Chaque endpoint définit explicitement :

- qui peut lire ;
- qui peut créer ;
- qui peut modifier ;
- qui peut supprimer.

Les contrôles d'accès sont obligatoires.

11. Versionnement

Les APIs doivent être conçues pour évoluer sans casser les clients existants.

Toute modification incompatible doit entraîner une nouvelle version.

Les changements rétrocompatibles sont privilégiés.

12. Pagination

Les collections importantes doivent être paginées.

Les APIs doivent supporter :

- pagination ;
 - tri ;
 - filtrage ;
 - recherche lorsque nécessaire.
-

13. Performance

Les APIs doivent :

- limiter le nombre de requêtes ;
 - éviter les traitements inutiles ;
 - utiliser les index disponibles ;
 - retourner uniquement les données nécessaires.
-

14. Journalisation

Les opérations critiques doivent être enregistrées.

Exemples :

- authentification ;
 - création UDI ;
 - modification Trust ;
 - synchronisation ;
 - changements sensibles.
-

15. Sécurité

Toutes les APIs doivent appliquer :

- validation des entrées ;
 - contrôle des permissions ;
 - politiques RLS ;
 - limitation des abus lorsque nécessaire ;
 - protection des données sensibles.
-

16. Tests

Chaque endpoint doit disposer de tests couvrant :

- succès ;
 - erreurs de validation ;
 - erreurs d'autorisation ;
 - erreurs d'authentification ;
 - cas limites.
-

17. Documentation

Chaque API doit être documentée avec :

- objectif ;
- endpoint ;
- méthode HTTP ;
- paramètres ;
- exemples de requêtes ;
- exemples de réponses ;
- codes d'erreur.

La documentation doit rester synchronisée avec l'implémentation.

18. Interdictions

Il est interdit :

- d'accéder directement à la base de données depuis l'UI ;
 - d'implémenter une logique métier dans les contrôleurs ;
 - de contourner les cas d'utilisation ;
 - de créer des endpoints non documentés ;
 - de modifier un contrat sans mise à jour de la documentation.
-

19. Critères de conformité

Une API est conforme lorsque :

- son contrat officiel est respecté ;
 - les validations sont complètes ;
 - les permissions sont appliquées ;
 - les tests sont validés ;
 - la documentation est à jour.
-

20. Conclusion

BUILD-APIS.md constitue la référence officielle d'implémentation des APIs de Vitala.

Toutes les APIs présentes et futures doivent respecter ce document afin de garantir une plateforme cohérente, sécurisée, maintenable et compatible avec les standards définis par le Build Blueprint.